

Vision-Based Satellite Rendezvous & Debris Tracking Simulator

Project Blueprint – Mathematical Reference & Phase Roadmap

Bishwaswarup Nayak

Bishwaswarup Nayak

Bishwaswarup Nayak

Bishwaswarup Nayak

August 21, 2026

A closed-loop GNC simulation framework for autonomous inspection and non-cooperative space debris rendezvous using monocular vision, nonlinear Kalman filtering, and predictive control.

Target Applications: TU Delft MSc Space Systems Engineering, NTU MAE, University of Tokyo / Tokyo Tech

Project Phases: 7 | **Primary Language:** C++17 + Python 3.11 | **License:** MIT

0 Contents

1	Project Overview & Motivation	3
2	Project Phase Roadmap	3
3	Phase 1 – Orbital Dynamics Engine	5
3.1	Reference Frames	5
3.2	Hill–Clohessy–Wiltshire (HCW) Equations	5
3.3	Yamanaka–Ankersen State Transition Matrix (YA-STM)	5
3.4	J_2 Perturbation	6
3.5	Phase 1 Asset Summary	6
4	Phase 2 – Target Kinematics & Tumbling Model	6
4.1	Rigid-Body Rotational Dynamics – Euler’s Equations	7
4.2	Attitude Representation – Unit Quaternion	7
4.3	Phase 2 Asset Summary	7
5	Phase 3 – Synthetic Vision Pipeline	7
5.1	Pinhole Camera Model	8
5.2	Image Noise Model	8
5.3	Feature Extraction Pipeline	8
5.4	Phase 3 Asset Summary	8
6	Phase 4 – Pose Estimation (PnP Solver)	9
6.1	The Perspective- n -Point Problem	9
6.2	EPnP Algorithm (Efficient PnP)	9
6.3	RANSAC Robustification	9
6.4	Phase 4 Asset Summary	10
7	Phase 5 – State Estimation: EKF & UKF	10
7.1	State & Measurement Vectors	10
7.2	Extended Kalman Filter (EKF)	10
7.3	Unscented Kalman Filter (UKF)	11
7.4	Noise Covariance Matrices	11
7.5	Phase 5 Asset Summary	11
8	Phase 6 – GNC: Guidance, Navigation & Control	12
8.1	Linear Quadratic Regulator (LQR)	12
8.2	Discrete-Time Model Predictive Control (MPC)	12
8.3	Δv Budget Analysis	13
8.4	Phase 6 Asset Summary	13
9	Phase 7 – Closed-Loop Integration & Validation	13
9.1	Closed-Loop Simulation Block Diagram	13
9.2	Monte Carlo Performance Metrics	13
9.3	Unit Test Suite	14
10	Complete Asset & Dependency Register	14

11 Key Mathematical Symbols Reference	15
12 Repository Structure	15
13 Expected Results & Publication-Grade Benchmarks	16
14 Conclusion & Next Steps	17

1 Project Overview & Motivation

Space debris in Low Earth Orbit (LEO) represents one of the most pressing engineering challenges of the 21st century. The Kessler syndrome, wherein collisions generate more debris in a self-sustaining cascade, threatens the long-term viability of orbital operations. Autonomous rendezvous and proximity operations (RPO) with non-cooperative, tumbling debris demands a tightly integrated Guidance, Navigation, and Control (GNC) architecture that can function without external beacons, GPS, or cooperative response from the target.

This project constructs a **flight-grade, closed-loop simulation environment** that:

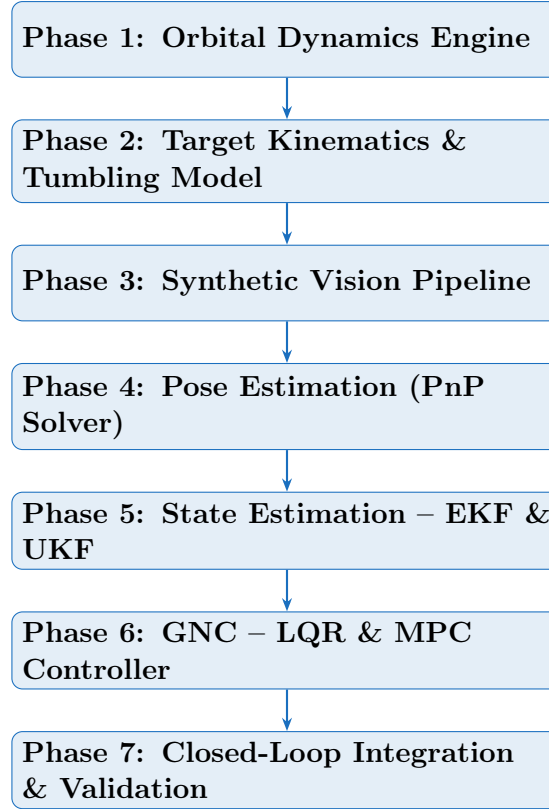
1. Propagates the relative orbital dynamics of a chaser satellite approaching an uncontrolled tumbling debris object.
2. Synthesizes realistic monocular camera feeds of the target at each time step.
3. Solves the 6-DOF pose estimation problem from 2D image keypoints using the Perspective- n -Point (PnP) algorithm.
4. Fuses noisy visual measurements with orbital dynamics priors using Extended and Unscented Kalman Filters.
5. Closes the loop with a Model Predictive Controller (MPC) that computes fuel-optimal thrust sequences.

Key Research Connections

- **TU Delft:** SpaceFlight lab (debris RPO), ADCS group (attitude estimation), AE4-872 Orbital Mechanics track
- **NTU:** MAE GNC research group, computational vision & robotics
- **Tokyo / ISAS:** Hayabusa-lineage GNC, proximity operations for asteroid/debris targets

2 Project Phase Roadmap

The project is structured into seven sequential but iteratively refinable phases. Each phase produces a testable, stand-alone software module.



2.0 Phase Deliverable Summary

Phase	Core Deliverable	Key Algorithms / Assets
1	HCW + YA propagator, J2 model	HCW ODEs, Yamanaka-Ankersen STM
2	Rigid-body attitude simulator	Euler's rotation equations, quaternion kinematics
3	Synthetic camera + renderer	Pinhole model, OpenCV, Blender API
4	6-DOF pose from images	EPnP, ORB/ArUco feature extraction
5	EKF & UKF state estimator	Jacobian propagation, sigma-point transform
6	LQR + MPC thrust planner	DARE solver, OSQP / CasADi
7	Full integrated simulator + Monte Carlo	GoogleTest, Matplotlib 3D, statistical analysis

3 Phase 1 – Orbital Dynamics Engine

Phase 1: Orbital Dynamics Engine

Goal: Build the numerical backbone that propagates the relative position and velocity of the chaser with respect to the tumbling target in the Local-Vertical Local-Horizontal (LVLH) frame.

Outputs: `dynamics/hcw.py`, `dynamics/ya_stm.cpp`, `dynamics/j2_perturb.cpp`

3.1 Reference Frames

- **ECI (Earth-Centred Inertial):** Origin at Earth’s CoM, axes fixed to distant stars. Used for absolute orbit propagation.
- **LVLH:** Origin at the chief (target) satellite. $\hat{x}$ along the radial direction, $\hat{z}$ along orbit normal, $\hat{y} = \hat{z} \times \hat{x}$ (along-track).

3.2 Hill–Clohessy–Wiltshire (HCW) Equations

For a chaser in a *near-circular* orbit about a chief, the linearised relative equations of motion in the LVLH frame are:

HCW Equations of Relative Motion

$$\ddot{x} - 2n\dot{y} - 3n^2x = f_x \quad (1)$$

$$\ddot{y} + 2n\dot{x} = f_y \quad (2)$$

$$\ddot{z} + n^2z = f_z \quad (3)$$

where $n = \sqrt{\mu/a^3}$ is the mean motion of the chief orbit, $\mu = 3.986004418 \times 10^{14} \text{ m}^3 \text{ s}^{-2}$ is Earth’s gravitational parameter, a is the chief semi-major axis, and f_x, f_y, f_z are the control accelerations per unit mass on the chaser.

The HCW state vector is $\mathbf{x}_{\text{HCW}} = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^\top \in \mathbb{R}^6$. The system matrix is:

$$\mathbf{A}_{\text{HCW}} = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 3n^2 & 0 & 0 & 0 & 2n & 0 \\ 0 & 0 & 0 & -2n & 0 & 0 \\ 0 & 0 & -n^2 & 0 & 0 & 0 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{0}_{3 \times 3} \\ \mathbf{I}_{3 \times 3} \end{pmatrix} \quad (4)$$

3.3 Yamanaka–Ankersen State Transition Matrix (YA-STM)

For *elliptic* orbits (eccentricity $e > 0$), the HCW model loses accuracy. The Yamanaka–Ankersen STM provides an analytic solution valid for any eccentricity:

YA State Transition Matrix

$$\mathbf{x}(f) = \Phi(f, f_0) \mathbf{x}(f_0) \quad (5)$$

where f is true anomaly and the STM $\Phi(f, f_0) \in \mathbb{R}^{6 \times 6}$ is constructed from the fundamental solution matrix:

$$\Phi(f, f_0) = \mathbf{M}(f) \mathbf{M}^{-1}(f_0) \quad (6)$$

with $\rho = 1 + e \cos f$, $s = \rho \sin f$, $c = \rho \cos f$, and $J = \int_{f_0}^f \frac{df'}{\rho^2(f')}$ (computed numerically).

The columns of $\mathbf{M}(f)$ are the six particular solutions of the homogeneous relative motion equations in the eccentric orbit case.

3.4 J_2 Perturbation

The dominant gravitational perturbation for LEO is the oblateness of Earth ($J_2 = 1.08263 \times 10^{-3}$). Its acceleration in ECI coordinates is:

J_2 Perturbing Acceleration

$$\mathbf{a}_{J_2} = \frac{3\mu J_2 R_E^2}{2r^5} \begin{pmatrix} x \left(5 \frac{z^2}{r^2} - 1 \right) \\ y \left(5 \frac{z^2}{r^2} - 1 \right) \\ z \left(5 \frac{z^2}{r^2} - 3 \right) \end{pmatrix} \quad (7)$$

where $R_E = 6,371$ km, $r = \|\mathbf{r}\|$.

3.5 Phase 1 Asset Summary

Phase 1 Assets

- **Libraries:** NumPy, SciPy (solve_ivp RK45/DOP853), Eigen3 (C++)
- **Constants:** μ , J_2 , R_E , ω_E (Earth rotation rate)
- **Test:** HCW drift-free periodic orbit verification, energy conservation check
- **Output:** Time-series $[\mathbf{r}(t), \mathbf{v}(t)]$ in LVLH frame

4 Phase 2 – Target Kinematics & Tumbling Model

Phase 2: Target Kinematics & Tumbling

Goal: Simulate the uncontrolled attitude dynamics of a rigid debris body (e.g., an upper rocket stage) including torque-free tumbling.

Outputs: target/attitude.py, target/rigid_body.cpp

4.1 Rigid-Body Rotational Dynamics – Euler’s Equations

Euler’s Equations (Body Frame)

$$I_1 \dot{\omega}_1 = (I_2 - I_3) \omega_2 \omega_3 + \tau_1 \quad (8)$$

$$I_2 \dot{\omega}_2 = (I_3 - I_1) \omega_3 \omega_1 + \tau_2 \quad (9)$$

$$I_3 \dot{\omega}_3 = (I_1 - I_2) \omega_1 \omega_2 + \tau_3 \quad (10)$$

For torque-free tumbling debris, $\boldsymbol{\tau} = \mathbf{0}$. $\mathbf{I} = \text{diag}(I_1, I_2, I_3)$ is the principal-axis inertia tensor.

4.2 Attitude Representation – Unit Quaternion

A unit quaternion $\mathbf{q} = [q_0, \mathbf{q}_v]^\top$ with $q_0 \in \mathbb{R}$ and $\mathbf{q}_v \in \mathbb{R}^3$, $\|\mathbf{q}\| = 1$, parameterises the rotation matrix as:

$$\mathbf{C}(\mathbf{q}) = (q_0^2 - \|\mathbf{q}_v\|^2) \mathbf{I}_3 + 2\mathbf{q}_v \mathbf{q}_v^\top + 2q_0 [\mathbf{q}_v]_\times \quad (11)$$

where $[\cdot]_\times$ is the skew-symmetric cross-product matrix.

Quaternion Kinematics

$$\dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\Xi}(\mathbf{q}) \boldsymbol{\omega}, \quad \boldsymbol{\Xi}(\mathbf{q}) = \begin{pmatrix} -\mathbf{q}_v^\top \\ q_0 \mathbf{I}_3 + [\mathbf{q}_v]_\times \end{pmatrix} \in \mathbb{R}^{4 \times 3} \quad (12)$$

The combined attitude-rate state is $\mathbf{x}_{\text{att}} = [\mathbf{q}^\top, \boldsymbol{\omega}^\top]^\top \in \mathbb{R}^7$.

4.3 Phase 2 Asset Summary

Phase 2 Assets

- Inertia tensor $\mathbf{I}$ for a representative cylindrical debris body (e.g., Ariane upper stage geometry)
- Quaternion normalisation constraint enforcement
- Simulation of Euler angles, Polhode trajectory (torque-free)
- Libraries: `scipy.integrate`, `Eigen3`, `pyquaternion`

5 Phase 3 – Synthetic Vision Pipeline

Phase 3: Synthetic Vision Pipeline

Goal: Generate realistic monocular grayscale/RGB images of the tumbling target at each simulation time step, with controllable camera noise, sensor blur, and lighting.

Outputs: `vision/camera.py`, `vision/renderer.py`,
`vision/feature_extractor.py`

5.1 Pinhole Camera Model

Pinhole Projection

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K} [\mathbf{R}_c \mid \mathbf{t}_c] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} \quad (13)$$

Camera intrinsic matrix:

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \quad (14)$$

where $f_x = f/d_x$, $f_y = f/d_y$ are the focal lengths in pixels, (c_x, c_y) is the principal point, and λ is the depth scale factor. $(\mathbf{R}_c, \mathbf{t}_c)$ are the camera extrinsic rotation and translation.

5.2 Image Noise Model

Each synthesised pixel intensity $I(u, v)$ is corrupted by additive white Gaussian noise (AWGN) and a Poisson shot noise term:

$$\tilde{I}(u, v) = I(u, v) + \mathcal{N}(0, \sigma_{\text{read}}^2) + \text{Poisson}(I(u, v)/g) \quad (15)$$

where g is the sensor gain factor. Default parameters: $\sigma_{\text{read}} = 2 \text{ DN}$, $g = 4 \text{ e}^-/\text{DN}$.

5.3 Feature Extraction Pipeline

Method	Use Case	Library
ArUco Markers	Known-target simulation / ground truth	<code>cv2.aruco</code>
ORB	Textured surfaces, real-imagery testing	<code>cv2.ORB</code>
Keypoint NN	Deep learning pose estimation (Phase 4+)	PyTorch

5.4 Phase 3 Asset Summary

Phase 3 Assets

- **3D Mesh:** Blender ‘.obj’ model of representative debris (Ariane 44L upper stage, or custom cylinder)
- **Renderer:** Blender Python API (bpy) or PyRenderer / ModernGL for real-time frames
- **Calibration:** Synthetic camera with $f = 50 \text{ mm}$, $1024 \times 1024 \text{ px}$, $\sigma_{\text{px}} = 2 \text{ px}$ noise

- **Dataset:** Generate $> 10,000$ labelled pose-image pairs for NN training (optional Phase 4 extension)

6 Phase 4 – Pose Estimation (PnP Solver)

Phase 4: Visual Pose Estimation

Goal: From a set of 2D image keypoint correspondences to known 3D object body-frame points, recover the 6-DOF relative pose $(\mathbf{R}, \mathbf{t})$ of the target in the camera frame.

Outputs: estimation/pnp_solver.py, estimation/epnp.cpp

6.1 The Perspective- n -Point Problem

Given n correspondences $\{(\mathbf{p}_i, \mathbf{u}_i)\}_{i=1}^n$ where $\mathbf{p}_i \in \mathbb{R}^3$ are body-frame 3D control points and $\mathbf{u}_i = (u_i, v_i)$ are image-plane pixel coordinates, find $(\mathbf{R}, \mathbf{t})$ minimising the reprojection error:

PnP Reprojection Objective

$$\min_{\mathbf{R} \in SO(3), \mathbf{t} \in \mathbb{R}^3} \sum_{i=1}^n \|\mathbf{u}_i - \pi(\mathbf{K}(\mathbf{R} \mathbf{p}_i + \mathbf{t}))\|^2 \quad (16)$$

where the projection $\pi([X, Y, Z]^\top) = [X/Z, Y/Z]^\top$.

6.2 EPnP Algorithm (Efficient PnP)

EPnP (Lepetit et al., 2009) expresses the n 3D points as weighted combinations of 4 *control points* $\mathbf{c}_j$:

$$\mathbf{p}_i = \sum_{j=1}^4 \alpha_{ij} \mathbf{c}_j, \quad \sum_{j=1}^4 \alpha_{ij} = 1 \quad (17)$$

This reduces the PnP problem to solving a 12×12 eigenvalue system, achieving $O(n)$ complexity with sub-pixel accuracy.

6.3 RANSAC Robustification

To handle feature mismatches and image clutter, EPnP is embedded in a RANSAC loop:

Algorithm 1 RANSAC-EPnP Pose Estimation

```

1: for  $k = 1, \dots, K_{\max}$  do
2:   Sample 4 random correspondences
3:   Solve EPnP  $\rightarrow (\mathbf{R}_k, \mathbf{t}_k)$ 
4:   Count inliers:  $|\{i : \text{reproj error}_i < \epsilon\}|$ 
5: end for
6: Return hypothesis with most inliers; refine with Levenberg-Marquardt

```

6.4 Phase 4 Asset Summary

Phase 4 Assets

- `cv2.solvePnP` with `SOLVEPNP_EPNP` flag (baseline)
- Custom C++ EPnP for latency benchmarking
- Accuracy target: rotation error $< 1.5^\circ$, translation error < 3 cm at 50 m range
- Ground-truth pose from Phase 1 & 2 for error evaluation

7 Phase 5 – State Estimation: EKF & UKF

Phase 5: Nonlinear State Estimation

Goal: Fuse noisy PnP pose measurements with the orbital dynamics model to reconstruct the full 12-state relative navigation state $\mathbf{x} = [\mathbf{r}, \mathbf{v}, \mathbf{q}, \boldsymbol{\omega}]^\top \in \mathbb{R}^{13}$ (with quaternion unit-norm constraint).

Outputs: `estimation/ekf.py`, `estimation/ukf.py`, `estimation/ekf.cpp`

7.1 State & Measurement Vectors

$$\mathbf{x}_k = \underbrace{\begin{pmatrix} \mathbf{r}_k \\ \mathbf{v}_k \end{pmatrix}}_{\text{translational}} \oplus \underbrace{\begin{pmatrix} \mathbf{q}_k \\ \boldsymbol{\omega}_k \end{pmatrix}}_{\text{rotational}} \in \mathbb{R}^6 \oplus \mathbb{R}^7 \quad (18)$$

The measurement vector from the PnP solver: $\mathbf{z}_k = [\hat{\mathbf{r}}_k^\top, \hat{\mathbf{q}}_k^\top]^\top \in \mathbb{R}^7$

7.2 Extended Kalman Filter (EKF)

EKF Predict–Update Cycle

Predict:

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}) \quad (19)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_{k-1} \mathbf{P}_{k-1|k-1} \mathbf{F}_{k-1}^\top + \mathbf{Q} \quad (20)$$

Update:

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R} \quad (21)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top \mathbf{S}_k^{-1} \quad (22)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})) \quad (23)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \quad (24)$$

Jacobians: $\mathbf{F}_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k}}$, $\mathbf{H}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k-1}}$

7.3 Unscented Kalman Filter (UKF)

The UKF avoids linearisation by propagating a set of $2L + 1$ deterministically chosen *sigma points* through the nonlinear functions, where $L = \dim(\mathbf{x})$.

Merwe-Scaled Sigma Point Selection

$$\boldsymbol{\sigma}_0 = \hat{\mathbf{x}} \quad (25)$$

$$\boldsymbol{\sigma}_i = \hat{\mathbf{x}} + \left(\sqrt{(L + \lambda) \mathbf{P}} \right)_i, \quad i = 1, \dots, L \quad (26)$$

$$\boldsymbol{\sigma}_{i+L} = \hat{\mathbf{x}} - \left(\sqrt{(L + \lambda) \mathbf{P}} \right)_i, \quad i = 1, \dots, L \quad (27)$$

Scaling: $\lambda = \alpha^2(L + \kappa) - L$ with $\alpha = 10^{-3}$, $\kappa = 0$, $\beta = 2$ (Gaussian prior assumption).

7.4 Noise Covariance Matrices

$$\mathbf{Q} = \text{diag}(\sigma_r^2 \mathbf{I}_3, \sigma_v^2 \mathbf{I}_3, \sigma_q^2 \mathbf{I}_4, \sigma_\omega^2 \mathbf{I}_3) \quad (28)$$

$$\mathbf{R} = \text{diag}(\sigma_{\text{pos, meas}}^2 \mathbf{I}_3, \sigma_{\text{att, meas}}^2 \mathbf{I}_4) \quad (29)$$

Typical LEO tuning values: $\sigma_r = 0.01$ m, $\sigma_v = 0.001$ m/s, $\sigma_q = 10^{-4}$, $\sigma_{\text{pos, meas}} = 0.05$ m (from PnP noise level).

7.5 Phase 5 Asset Summary

Phase 5 Assets

- Python: `filterpy` library (EKF, UKF baseline)
- C++: Custom EKF with Eigen3 matrix operations
- Quaternion EKF: Multiplicative EKF (MEKF) to respect $SO(3)$ manifold
- Performance metric: 3- σ consistency test (NIS/NEES)

8 Phase 6 – GNC: Guidance, Navigation & Control

Phase 6: Closed-Loop Control

Goal: Using the estimated state from Phase 5, compute an optimal thrust sequence that drives the chaser to the desired hold-point (e.g., 10 m stand-off) while minimising propellant consumption.

Outputs: control/lqr.py, control/mpc.py

8.1 Linear Quadratic Regulator (LQR)

For the linearised HCW system $(\mathbf{A}_{\text{HCW}}, \mathbf{B})$, the LQR gain matrix $\mathbf{K}_{\text{LQR}}$ minimises the infinite-horizon cost:

LQR Cost Function

$$J_{\text{LQR}} = \int_0^\infty [\mathbf{x}^\top \mathbf{Q}_c \mathbf{x} + \mathbf{u}^\top \mathbf{R}_c \mathbf{u}] dt \quad (30)$$

Optimal gain: $\mathbf{K}_{\text{LQR}} = \mathbf{R}_c^{-1} \mathbf{B}^\top \mathbf{P}^*$
where $\mathbf{P}^*$ solves the Algebraic Riccati Equation (ARE):

$$\mathbf{A}^\top \mathbf{P}^* + \mathbf{P}^* \mathbf{A} - \mathbf{P}^* \mathbf{B} \mathbf{R}_c^{-1} \mathbf{B}^\top \mathbf{P}^* + \mathbf{Q}_c = \mathbf{0} \quad (31)$$

Control law: $\mathbf{u}^*(t) = -\mathbf{K}_{\text{LQR}} \mathbf{x}(t)$.

8.2 Discrete-Time Model Predictive Control (MPC)

MPC optimises a finite receding-horizon trajectory subject to constraints. At each time step k , solve:

MPC Optimisation Problem

$$\min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \sum_{j=0}^{N-1} [\mathbf{x}_j^\top \mathbf{Q}_c \mathbf{x}_j + \mathbf{u}_j^\top \mathbf{R}_c \mathbf{u}_j] + \mathbf{x}_N^\top \mathbf{P}^* \mathbf{x}_N \quad (32)$$

Subject to:

$$\mathbf{x}_{j+1} = \mathbf{A}_d \mathbf{x}_j + \mathbf{B}_d \mathbf{u}_j \quad (\text{discrete HCW}) \quad (33)$$

$$\|\mathbf{u}_j\|_2 \leq u_{\max} \quad (\text{thrust limit}) \quad (34)$$

$$\sum_j \|\mathbf{u}_j\|_1 \leq \Delta v_{\text{budget}} \quad (\text{fuel budget}) \quad (35)$$

The problem is a quadratic program (QP); solved with OSQP at ~ 100 Hz onboard control loop frequency.

Discretisation ($T_s = \text{sample period}$):

$$\mathbf{A}_d = e^{\mathbf{A}_{\text{HCW}} T_s}, \quad \mathbf{B}_d = \left(\int_0^{T_s} e^{\mathbf{A}_{\text{HCW}} \tau} d\tau \right) \mathbf{B} \quad (36)$$

8.3 Δv Budget Analysis

Total propellant consumed during a rendezvous sequence of M manoeuvres:

$$\Delta v_{\text{total}} = \sum_{k=1}^M \|\Delta \mathbf{v}_k\|_2, \quad m_{\text{prop}} = m_0 \left(1 - e^{-\Delta v_{\text{total}} / (I_{\text{sp}} g_0)}\right) \quad (37)$$

8.4 Phase 6 Asset Summary

Phase 6 Assets

- **LQR:** `scipy.linalg.solve_continuous_are` / `control.lqr`
- **MPC:** `osqp-python`, `CasADi` (for nonlinear MPC extension)
- **Constraints:** $u_{\text{max}} = 1$ N (cold-gas thruster budget), $\Delta v \leq 20$ m/s total
- **Benchmark:** MPC vs PD controller Δv comparison

9 Phase 7 – Closed-Loop Integration & Validation

Phase 7: System Integration & Monte Carlo Validation

Goal: Integrate all six prior modules into a single simulation loop and perform statistical validation through Monte Carlo analysis.

Outputs: `sim/main_loop.py`, `tests/`, plots, final benchmark table

9.1 Closed-Loop Simulation Block Diagram

At each time step k of the simulation:

1. **Propagate:** Phase 1 $\rightarrow$ update $[\mathbf{r}_k, \mathbf{v}_k]_{\text{LVLH}}$
2. **Attitude:** Phase 2 $\rightarrow$ update $[\mathbf{q}_k, \boldsymbol{\omega}_k]$ of target
3. **Render:** Phase 3 $\rightarrow$ generate synthetic camera frame I_k
4. **Detect:** Phase 3 $\rightarrow$ extract keypoints/features from I_k
5. **PnP:** Phase 4 $\rightarrow$ recover noisy pose measurement $\mathbf{z}_k$
6. **Filter:** Phase 5 $\rightarrow$ EKF/UKF update $\rightarrow \hat{\mathbf{x}}_k$
7. **Control:** Phase 6 $\rightarrow$ MPC solve $\rightarrow \mathbf{u}_k^*$
8. **Apply:** Thrust $\mathbf{u}_k^*$ feeds back into Phase 1 as f_x, f_y, f_z

9.2 Monte Carlo Performance Metrics

Run $N_{\text{MC}} = 500$ trials with randomly sampled:

- Initial relative position: $\mathbf{r}_0 \sim \mathcal{U}([50, 200] \text{ m})$

- Camera pixel noise: $\sigma_{\text{px}} \sim \mathcal{U}(0.5, 3.0)$ px
- Target tumble rate: $\omega_0 \sim \mathcal{U}(0.5, 5.0)$ deg/s

Metric	Definition	Target	Units
Position Error	$\ \hat{\mathbf{r}} - \mathbf{r}\ _2$	< 0.05	m
Velocity Error	$\ \hat{\mathbf{v}} - \mathbf{v}\ _2$	< 0.01	m/s
Attitude Error	$2 \arccos(\hat{q}_0 q_0 + \hat{\mathbf{q}}_v \cdot \mathbf{q}_v)$	< 1.2	deg
Δv Consumed	$\sum \ \Delta \mathbf{v}\ $ (MPC vs LQR)	14% saving	m/s
PnP Latency	Time per frame	< 10	ms
Filter NIS	Normalised innovation	$\chi^2_{n_z, 95\%}$	—

9.3 Unit Test Suite

- HCW drift-free orbit: 5-orbit propagation without divergence
- EKF consistency: NIS χ^2 test at 95% confidence
- PnP accuracy: sub-pixel reprojection error on synthetic data
- MPC feasibility: QP solved in < 10 ms, constraints satisfied

10 Complete Asset & Dependency Register

Category	Tool / Library	Purpose
— Core Languages —		
Language	Python 3.11	Simulation, data pipeline, visualisation
Language	C++17	EKF/UKF core, EPnP, performance-critical code
Build	CMake 3.25	C++ build system
— Numerical / Scientific —		
NumPy	1.26	Array ops, linear algebra
SciPy	1.12	ODE integration (RK45, DOPRI853), ARE solver
Eigen3	3.4	C++ matrix library
Matplotlib	3.8	2D/3D plotting, Monte Carlo histograms
— Computer Vision —		
OpenCV	4.9	Camera model, PnP, ORB/ArUco, RANSAC
Blender Python API	4.x	3D mesh rendering (offline frames)
PyTorch	2.2	Keypoint NN (optional Phase 4 extension)
— Estimation & Control —		
filterpy	1.4	EKF, UKF reference implementation
python-control	0.9	LQR, ARE, Bode/root-locus analysis

Category	Tool / Library	Purpose
CasADi	3.6	Symbolic differentiation, NLP/MPC formulation
OSQP	0.6	Fast QP solver for MPC
— <i>Astrodynamics</i> —		
Astropy	6.0	Time conversions, coordinate frames
Poliastro	0.17	Orbit propagation baseline comparison
— <i>Testing & CI</i> —		
GoogleTest	1.14	C++ unit tests
pytest	8.0	Python unit/integration tests
GitHub Actions	—	CI pipeline (build, test, lint)

11 Key Mathematical Symbols Reference

Symbol	Definition	Units
n	Mean orbital motion of chief, $n = \sqrt{\mu/a^3}$	rad/s
μ	Earth gravitational parameter	$\text{m}^3 \text{s}^{-2}$
a	Chief orbit semi-major axis	m
e	Orbital eccentricity	—
J_2	Earth oblateness coefficient	—
$\mathbf{r}$	Relative position in LVLH	m
$\mathbf{v}$	Relative velocity in LVLH	m/s
$\mathbf{q}$	Unit quaternion (target attitude)	—
$\boldsymbol{\omega}$	Angular velocity (target body frame)	rad/s
$\mathbf{I}$	Principal inertia tensor	$\text{kg}\cdot\text{m}^2$
$\mathbf{K}$	Camera intrinsic matrix	px
$\mathbf{R}_c, \mathbf{t}_c$	Camera extrinsic rotation & translation	—/m
$\mathbf{P}$	State error covariance matrix	(state units) ²
$\mathbf{Q}$	Process noise covariance	(state units) ²
$\mathbf{R}$	Measurement noise covariance	(meas units) ²
$\mathbf{K}$	Kalman gain matrix	—
$\mathbf{Q}_c, \mathbf{R}_c$	LQR/MPC cost weight matrices	—
Δv	Delta-velocity manoeuvre magnitude	m/s
I_{sp}	Engine specific impulse	s
N	MPC prediction horizon	steps
T_s	Discrete control sample period	s

12 Repository Structure

```
vision-satellite-rendezvous/
|-- config/
|   |-- orbit_params.yaml           # Chief orbit, simulation duration
|   |-- camera_params.yaml         # Intrinsics, noise parameters
```

```

|   '-- control_params.yaml           # MPC horizon, LQR weights
|-- dynamics/
|   |-- hcw.py                       # Phase 1: HCW propagator
|   |-- ya_stm.cpp                   # Phase 1: YA state-transition matrix
|   '-- j2_perturb.cpp                # Phase 1: J2 acceleration
|-- target/
|   |-- attitude.py                  # Phase 2: Euler + quaternion kinematics
|   '-- rigid_body.cpp               # Phase 2: C++ rigid-body integrator
|-- vision/
|   |-- camera.py                    # Phase 3: Pinhole model, distortion
|   |-- renderer.py                  # Phase 3: Blender/OpenCV renderer
|   '-- feature_extractor.py         # Phase 3: ORB, ArUco, keypoint NN
|-- estimation/
|   |-- pnp_solver.py                # Phase 4: EPnP + RANSAC wrapper
|   |-- epnp.cpp                     # Phase 4: C++ EPnP core
|   |-- ekf.py                       # Phase 5: EKF (translational + MEKF)
|   '-- ukf.py                       # Phase 5: UKF with sigma points
|-- control/
|   |-- lqr.py                       # Phase 6: LQR design (ARE solver)
|   '-- mpc.py                       # Phase 6: OSQP-based MPC
|-- sim/
|   '-- main_loop.py                 # Phase 7: Closed-loop simulation entry
|-- tests/
|   |-- test_hcw.cpp                 # GoogleTest: HCW orbit conservation
|   |-- test_ekf.py                  # pytest: EKF NIS consistency
|   '-- test_pnp.py                  # pytest: PnP reprojection accuracy
|-- notebooks/
|   '-- phase_demo.ipynb             # Jupyter notebooks per phase
|-- CMakeLists.txt
|-- requirements.txt
'-- README.md

```

13 Expected Results & Publication-Grade Benchmarks

Result	Phase	Expected Value	Comparison
Steady-state position error	5	< 0.05 m	EKF vs UKF
Attitude estimation error	5	$< 1.2^\circ$	MEKF consistency
PnP rotation error	4	$< 1.5^\circ$	ORB vs ArUco
MPC Δv saving	6	$\approx 14\%$	MPC vs PD
MPC solve time	6	< 10 ms	OSQP hot-start
Monte Carlo pass rate	7	$> 95\%$	500 trials

14 Conclusion & Next Steps

This blueprint defines a complete, executable path from first principles to a publication-quality closed-loop space rendezvous simulator. Each of the seven phases is independently testable and produces a concrete artefact. The mathematical formulations are grounded in established aerospace literature (HCW, YA, EPnP, MEKF, MPC-OSQP) while the implementation stack (C++17 + Python 3.11, OpenCV, Eigen3, CasADi) reflects real-world flight-software practice.

Immediate next steps after this document:

1. Set up GitHub repository and CI skeleton
2. Implement and test Phase 1 (HCW + J2) with unit test coverage
3. Validate against Poliastro reference propagator
4. Begin Phase 2 attitude integrator in parallel

Vision-Based Satellite Rendezvous Simulator – Blueprint v1.0
Bishwaswarup Nayak · August 2026